Assignment 1 Deep Learning and Sensor fusion 2026

Multi class Classification

Necessary iomport here

In [1]:
```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
from torch.utils.data import Dataset
import torch.nn.functional as F
import matplotlib.pyplot as plt
from PIL import Image
from torchvision import transforms
import os
import numpy as np
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split

print(f"[INFO] Torch infos: {torch.__version__}")

DATASET_PATH = os.path.join("..", "Dataset", "BoneBreakClassification")
```

```
[INFO] Torch infos: 2.9.1+cu128
```

In [2]:
```
# some helpers
def load_data(dataset_path, class_names, type_of_data_to_load):
    # for each class name
    images, labels = list(), list()
    for el in class_names.keys():
        print(f"[WALKING] Walking into {el}")
        for images_name in os.listdir(os.path.join(dataset_path, el, type
            full_image_path = os.path.join(dataset_path, el, type_of_data
            images.append(full_image_path)
            labels.append(class_names[el])
    return images, labels
```

set Dataset path

The path yopu see in the code point to my folder

```
.
├── Assignment_1
│   ├── Assignment_1_four_layers.ipynb
│   ├── Assignment_1_template.ipynb
│   ├── Assignment_1_three_layers.ipynb
│   └── cnn_ass_1.pt
├── Dataset
│   ├── BoneBreakClassification
```

If you put the data in a different place change it.

In [3]:
```
DATASET_PATH = os.path.join("Dataset","BoneBreakClassification")
```

(1) Load dataset

In [4]:
```
class_names = dict()

temp = os.listdir(DATASET_PATH)
temp = sorted(temp)

# prep up list and class number together
for i in range(len(temp)):
    class_names[temp[i]] = i

data_to_split, labels_to_split = load_data(dataset_path=DATASET_PATH,
                                                        class_names=class_
                                                        type_of_data_to_lo

print(f"[DATA TO SPLIT] Data loaded complete {len(data_to_split)} Labeles

# load test data
test_data_handler, test_labels_handler = load_data(dataset_path=DATASET_P
                                                    class_names=class_name
                                                    type_of_data_to_load="

print(f"[TRAIN DATA LOADED] Data loaded complete {len(test_data_handler)}
```

```
[WALKING] Walking into Avulsion fracture
[WALKING] Walking into Comminuted fracture
[WALKING] Walking into Fracture Dislocation
[WALKING] Walking into Greenstick fracture
[WALKING] Walking into Hairline Fracture
[WALKING] Walking into Impacted fracture
[WALKING] Walking into Longitudinal fracture
[WALKING] Walking into Oblique fracture
[WALKING] Walking into Pathological fracture
[WALKING] Walking into Spiral Fracture
[DATA TO SPLIT] Data loaded complete 989 Labeles are 989
[WALKING] Walking into Avulsion fracture
[WALKING] Walking into Comminuted fracture
[WALKING] Walking into Fracture Dislocation
[WALKING] Walking into Greenstick fracture
[WALKING] Walking into Hairline Fracture
[WALKING] Walking into Impacted fracture
[WALKING] Walking into Longitudinal fracture
[WALKING] Walking into Oblique fracture
[WALKING] Walking into Pathological fracture
[WALKING] Walking into Spiral Fracture
[TRAIN DATA LOADED] Data loaded complete 140 Labeles are 140
```

(2) leave this cell because we need to create a val dataset and it is not there as default

In [5]:
```
# here you can split validation in stead of test so you can preserve inta
train_data_handler, validation_data_handler, train_labels_handler, valida

print(f"[TRAIN DATA] The train data is {train_data_handler.shape}  and it
print(f"[VALIDATION DATA] The validation data is {validation_data_handler

# my data loader need list yours? adapt it to your code
train_data_handler = list(train_data_handler)
train_labels_handler = list(train_labels_handler)

validation_data_handler = list(validation_data_handler)
validation_labels_handler = list(validation_labels_handler)
```

```
[TRAIN DATA] The train data is (791,)  and its labels are (791,)
[VALIDATION DATA] The validation data is (198,)  and its labels are (198,)
```

(3) Here ate two transforms one for training and one for evaluation

train transform need data augmentation:

- add random flip
- add random rotation
- add color jitter

both need need:

- resize

- toTensor handler
- Normalize (use the provided MEAN and STD)

Note that the evaluation transform does not need data augmentation.

In [6]:
```
TARGET_H = 224
TARGET_W = 224
IMAGE_MEAN = [0.485,0.456,0.406]
IMAGE_STD = [0.229,0.224,0.225]

rotation_degree = 20
# create transform
train_transform = transforms.Compose([
    transforms.Resize((TARGET_W,TARGET_H)),
    transforms.RandomVerticalFlip(),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(rotation_degree),
    transforms.ColorJitter(),
    transforms.ToTensor(),
    transforms.Normalize(IMAGE_MEAN,IMAGE_STD)
])

evaluation_transform = transforms.Compose([
    transforms.Resize((TARGET_W,TARGET_H)),
    transforms.ToTensor(),
    transforms.Normalize(IMAGE_MEAN,IMAGE_STD)
])
```

In [7]:
```
class MyDataset(Dataset):
    def __init__(self, list_of_pathes, list_of_classes, transform=None) -
        super().__init__()
        self.list_of_pathes = list_of_pathes
        self.list_of_classes = list_of_classes
        self.transform = transform

    def __len__(self):
        return len(self.list_of_pathes)

    def __getitem__(self, index):
        path = self.list_of_pathes[index]
        label = self.list_of_classes[index]
        image = Image.open(path).convert("RGB")
        image_as_tensor= self.transform(image)
        # return image and label
        return image_as_tensor, label
```

(4) Create data loder from the MyDataset class

In [ ]:
```
train_dataset = MyDataset(list_of_pathes=train_data_handler,
                          list_of_classes=train_labels_handler,
                          transform=train_transform)

validation_dataset = MyDataset(list_of_pathes=validation_data_handler,
                               list_of_classes=validation_labels_handler
                               transform=evaluation_transform)

test_dataset = MyDataset(list_of_pathes=test_data_handler,
```

```
                            list_of_classes=test_labels_handler,
                            transform=evaluation_transform)

# put those here
train_loader = DataLoader(train_dataset,batch_size=64,shuffle=True,num_wo
val_loader = DataLoader(validation_dataset,batch_size=128,shuffle=False,
test_loader = DataLoader(test_dataset,batch_size=128,shuffle=True, num_wo
```

Instance of the CNN

Use the Sequential API it is faster

- This cnn has two hidden layers (conv2d -> proper activation -> max pool 2d) one has 16 untis and the second has 32 units
- Add a flatten layer
- Classifier need 128 Linear unit proper activation and set dropdown at 0.5
- The last layer of the classifier how many units need?

In [9]:
```
class MyCNN(nn.Module):
    def __init__(self,
                 number_of_output_classes,
                 kernel_size,
                 padding_size,
                 pooling_size,
                 input_channels,
                 dropout_probabilities: float,
                 dense_units: list[int],
                 layers_list: list[int]
                 ):

        super().__init__()

        layers = []

        input_units = input_channels

        for output_units in layers_list:
            layers +=[
                nn.Conv2d(input_units,output_units,kernel_size=kernel_siz
                nn.ReLU(inplace=True),
                nn.MaxPool2d(pooling_size)
            ]
            input_units = output_units

        self.features = nn.Sequential(*layers)

        mlp = []
        count_layer = 0
        for i in layers_list:
            count_layer +=1
        if count_layer == 2:
            in_features = layers_list[-1]*56*56 # since maxpool 2 times
        else:
            in_features = layers_list[-1]*28*28 # since maxpool 3 times


        for h in dense_units:
```

```
            mlp += [
                nn.Linear(in_features,h),
                nn.ReLU(inplace=True),
                nn.Dropout(p=dropout_probabilities)
            ]
            in_features = h

        self.classifier = nn.Sequential(*mlp)
        self.flatten_layer = nn.Flatten()
        self.final_classifier = nn.Linear(in_features,number_of_output_cl

    def forward(self,x):
        x = self.features(x)
        x = self.flatten_layer(x)
        x = self.classifier(x)
        x = self.final_classifier(x)

        return x
```

(5) Add proper optimizer and loss function set training epochs to 50

In [10]:
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print (f"Device is {device}")
```

```
Device is cuda
```

In [11]:
```
def train(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader :
        images,labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        logits = model(images)
        loss = criterion(logits,labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()*images.size(0)
        preds = logits.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    train_loss = running_loss / total
    accuracy = correct / total
    return train_loss, accuracy

params = {
    "number_of_output_classes":10,
    "kernel_size":3,
    "padding_size":1,
    "pooling_size":2,
    "input_channels":3,
    "dropout_probabilities":0.5,
    "dense_units":[128],
```

```
    "layers_list":[16,32]
}
model = MyCNN(**params)
model.to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(),lr= 1e-3,weight_decay=1e-4)
```

In [12]:
```
@torch.no_grad()
def evaluation(model,loader,criterion,device):
    model.eval()
    correct,total = 0,0
    running_loss = 0.0

    for image, label in loader:
        image = image.to(device)
        label = label.to(device)

        logits = model(image)
        loss= criterion(logits,label)
        current__image_loss = loss.item()

        running_loss += current__image_loss*image.size(0)

        prediction = logits.argmax(dim=1)
        correct += (prediction == label).sum().item()
        total += label.size(0)

    val_loss = running_loss / total
    val_accuracy = correct / total
    return val_loss, val_accuracy
```

(6) add here the train and evaluation loop collect loss and accuracy, we want to see curves here

In [13]:
```
epochs = 50
best_val_acc = 0
val_acc = 0
train_losses, train_accs = [], []
val_losses, val_accs = [], []

for epoch in range (1, epochs + 1):
    train_loss, train_acc = train(model = model,loader=train_loader, crit
    print(f"Epoch #{epoch}/{epochs}, Train Loss = {train_loss:.4f}, Accur
    train_losses.append(train_loss)
    train_accs.append(train_acc)

    val_loss, val_acc = evaluation(model = model,loader=val_loader, crite
    print(f"Epoch #{epoch}/{epochs}, Val Loss = {val_loss:.4f}, Val Accur
    val_losses.append(val_loss)
    val_accs.append(val_acc)

    if (val_acc > best_val_acc):
        best_val_acc = val_acc
        torch.save(model.state_dict(),"Assignment1_best_model_2layer.pt")
print("Best Val Accuracy", best_val_acc)
```

```
Epoch #1/50, Train Loss = 3.7388, Accuracy = 0.105
Epoch #1/50, Val Loss = 2.2940, Val Accuracy = 0.116
Epoch #2/50, Train Loss = 2.2893, Accuracy = 0.149
Epoch #2/50, Val Loss = 2.2899, Val Accuracy = 0.146
Epoch #3/50, Train Loss = 2.2759, Accuracy = 0.152
Epoch #3/50, Val Loss = 2.2665, Val Accuracy = 0.157
Epoch #4/50, Train Loss = 2.2456, Accuracy = 0.168
Epoch #4/50, Val Loss = 2.2404, Val Accuracy = 0.182
Epoch #5/50, Train Loss = 2.2559, Accuracy = 0.174
Epoch #5/50, Val Loss = 2.2407, Val Accuracy = 0.207
Epoch #6/50, Train Loss = 2.2422, Accuracy = 0.166
Epoch #6/50, Val Loss = 2.2339, Val Accuracy = 0.187
Epoch #7/50, Train Loss = 2.2057, Accuracy = 0.196
Epoch #7/50, Val Loss = 2.2035, Val Accuracy = 0.192
Epoch #8/50, Train Loss = 2.1753, Accuracy = 0.215
Epoch #8/50, Val Loss = 2.1963, Val Accuracy = 0.192
Epoch #9/50, Train Loss = 2.1540, Accuracy = 0.225
Epoch #9/50, Val Loss = 2.2163, Val Accuracy = 0.212
Epoch #10/50, Train Loss = 2.1601, Accuracy = 0.211
Epoch #10/50, Val Loss = 2.1917, Val Accuracy = 0.242
Epoch #11/50, Train Loss = 2.1342, Accuracy = 0.220
Epoch #11/50, Val Loss = 2.1618, Val Accuracy = 0.192
Epoch #12/50, Train Loss = 2.1339, Accuracy = 0.233
Epoch #12/50, Val Loss = 2.1587, Val Accuracy = 0.227
Epoch #13/50, Train Loss = 2.1477, Accuracy = 0.224
Epoch #13/50, Val Loss = 2.1705, Val Accuracy = 0.232
Epoch #14/50, Train Loss = 2.1162, Accuracy = 0.239
Epoch #14/50, Val Loss = 2.1749, Val Accuracy = 0.217
Epoch #15/50, Train Loss = 2.0964, Accuracy = 0.249
Epoch #15/50, Val Loss = 2.1725, Val Accuracy = 0.227
Epoch #16/50, Train Loss = 2.0703, Accuracy = 0.249
Epoch #16/50, Val Loss = 2.1838, Val Accuracy = 0.217
Epoch #17/50, Train Loss = 2.0494, Accuracy = 0.235
Epoch #17/50, Val Loss = 2.1716, Val Accuracy = 0.232
Epoch #18/50, Train Loss = 2.0310, Accuracy = 0.279
Epoch #18/50, Val Loss = 2.1711, Val Accuracy = 0.247
Epoch #19/50, Train Loss = 2.0494, Accuracy = 0.236
Epoch #19/50, Val Loss = 2.1503, Val Accuracy = 0.242
Epoch #20/50, Train Loss = 1.9709, Accuracy = 0.306
Epoch #20/50, Val Loss = 2.1692, Val Accuracy = 0.258
Epoch #21/50, Train Loss = 1.9605, Accuracy = 0.282
Epoch #21/50, Val Loss = 2.1661, Val Accuracy = 0.232
Epoch #22/50, Train Loss = 1.9909, Accuracy = 0.293
Epoch #22/50, Val Loss = 2.1904, Val Accuracy = 0.253
Epoch #23/50, Train Loss = 2.0022, Accuracy = 0.283
Epoch #23/50, Val Loss = 2.1852, Val Accuracy = 0.222
Epoch #24/50, Train Loss = 1.9728, Accuracy = 0.286
Epoch #24/50, Val Loss = 2.1751, Val Accuracy = 0.258
Epoch #25/50, Train Loss = 1.9272, Accuracy = 0.306
Epoch #25/50, Val Loss = 2.1633, Val Accuracy = 0.237
Epoch #26/50, Train Loss = 1.9303, Accuracy = 0.326
Epoch #26/50, Val Loss = 2.1807, Val Accuracy = 0.207
Epoch #27/50, Train Loss = 1.9497, Accuracy = 0.302
Epoch #27/50, Val Loss = 2.2206, Val Accuracy = 0.237
Epoch #28/50, Train Loss = 1.9686, Accuracy = 0.297
Epoch #28/50, Val Loss = 2.1651, Val Accuracy = 0.217
Epoch #29/50, Train Loss = 1.8908, Accuracy = 0.321
Epoch #29/50, Val Loss = 2.1787, Val Accuracy = 0.263
Epoch #30/50, Train Loss = 1.8798, Accuracy = 0.322
Epoch #30/50, Val Loss = 2.1932, Val Accuracy = 0.247
```

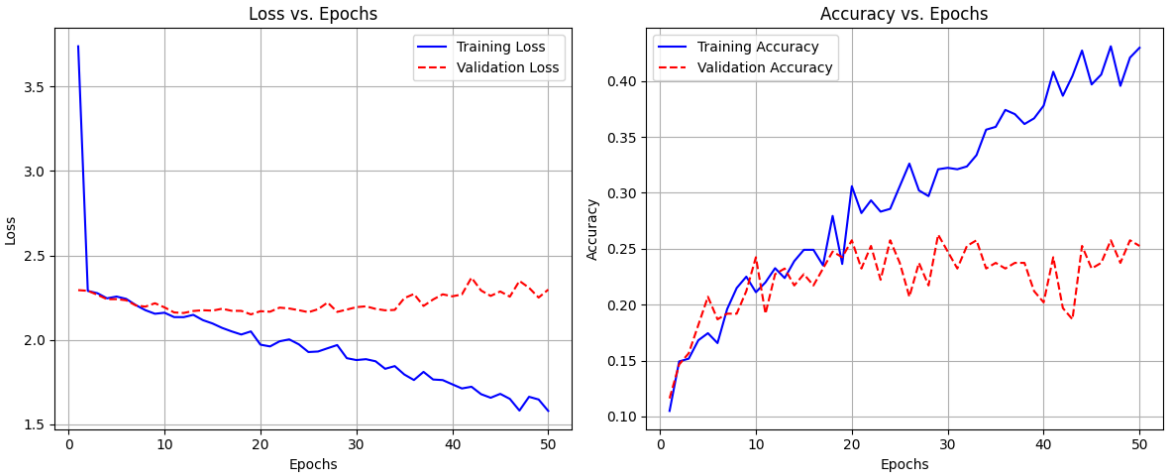
```
Epoch #31/50, Train Loss = 1.8849, Accuracy = 0.321
Epoch #31/50, Val Loss = 2.1981, Val Accuracy = 0.232
Epoch #32/50, Train Loss = 1.8725, Accuracy = 0.324
Epoch #32/50, Val Loss = 2.1829, Val Accuracy = 0.253
Epoch #33/50, Train Loss = 1.8279, Accuracy = 0.334
Epoch #33/50, Val Loss = 2.1745, Val Accuracy = 0.258
Epoch #34/50, Train Loss = 1.8439, Accuracy = 0.357
Epoch #34/50, Val Loss = 2.1769, Val Accuracy = 0.232
Epoch #35/50, Train Loss = 1.7940, Accuracy = 0.359
Epoch #35/50, Val Loss = 2.2466, Val Accuracy = 0.237
Epoch #36/50, Train Loss = 1.7614, Accuracy = 0.374
Epoch #36/50, Val Loss = 2.2711, Val Accuracy = 0.232
Epoch #37/50, Train Loss = 1.8097, Accuracy = 0.370
Epoch #37/50, Val Loss = 2.2003, Val Accuracy = 0.237
Epoch #38/50, Train Loss = 1.7645, Accuracy = 0.362
Epoch #38/50, Val Loss = 2.2386, Val Accuracy = 0.237
Epoch #39/50, Train Loss = 1.7611, Accuracy = 0.367
Epoch #39/50, Val Loss = 2.2692, Val Accuracy = 0.212
Epoch #40/50, Train Loss = 1.7359, Accuracy = 0.378
Epoch #40/50, Val Loss = 2.2572, Val Accuracy = 0.202
Epoch #41/50, Train Loss = 1.7114, Accuracy = 0.408
Epoch #41/50, Val Loss = 2.2685, Val Accuracy = 0.242
Epoch #42/50, Train Loss = 1.7216, Accuracy = 0.387
Epoch #42/50, Val Loss = 2.3679, Val Accuracy = 0.197
Epoch #43/50, Train Loss = 1.6776, Accuracy = 0.405
Epoch #43/50, Val Loss = 2.2917, Val Accuracy = 0.187
Epoch #44/50, Train Loss = 1.6564, Accuracy = 0.427
Epoch #44/50, Val Loss = 2.2597, Val Accuracy = 0.253
Epoch #45/50, Train Loss = 1.6795, Accuracy = 0.397
Epoch #45/50, Val Loss = 2.2861, Val Accuracy = 0.232
Epoch #46/50, Train Loss = 1.6496, Accuracy = 0.406
Epoch #46/50, Val Loss = 2.2554, Val Accuracy = 0.237
Epoch #47/50, Train Loss = 1.5804, Accuracy = 0.431
Epoch #47/50, Val Loss = 2.3486, Val Accuracy = 0.258
Epoch #48/50, Train Loss = 1.6620, Accuracy = 0.396
Epoch #48/50, Val Loss = 2.3078, Val Accuracy = 0.237
Epoch #49/50, Train Loss = 1.6460, Accuracy = 0.421
Epoch #49/50, Val Loss = 2.2499, Val Accuracy = 0.258
Epoch #50/50, Train Loss = 1.5783, Accuracy = 0.430
Epoch #50/50, Val Loss = 2.2971, Val Accuracy = 0.253
Best Val Accuracy 0.26262626262626265
```

In [14]:
```
model.load_state_dict(torch.load("Assignment1_best_model_2layer.pt"))
print(f"Loaded best model with validation accuracy: {best_val_acc:.4f}")
```

```
Loaded best model with validation accuracy: 0.2626
```

In [15]:
```
plt.figure(figsize=(12, 5))
# Plot 1: Loss Curves
plt.subplot(1, 2, 1)
plt.plot(range(1, len(train_losses) + 1), train_losses, label='Training L
plt.plot(range(1, len(val_losses) + 1), val_losses, label='Validation Los
plt.title('Loss vs. Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
# Plot 2: Accuracy Curves
plt.subplot(1, 2, 2)
plt.plot(range(1, len(train_accs) + 1), train_accs, label='Training Accur
```

```
plt.plot(range(1, len(val_accs) + 1), val_accs, label='Validation Accurac
plt.title('Accuracy vs. Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



(7) Make prediction and shows some predicted images

In [ ]:
```
def visualize_predictions(model, data_loader, device, num_images=8, figsi

    model.eval()
    data_iter = iter(data_loader)
    images, labels = next(data_iter)
    images, labels = images.to(device), labels.to(device)

    with torch.no_grad():
        logits = model(images)
        probs = F.softmax(logits, dim=1)
        predictions = torch.argmax(probs, dim=1)

    plt.figure(figsize=figsize)
    for i in range(num_images):
        plt.subplot(2, 4, i + 1)
        img = images[i].cpu().numpy().transpose((1, 2, 0))
        img = img * IMAGE_STD + IMAGE_MEAN
        img = np.clip(img, 0, 1)
        plt.imshow(img)

        gt = labels[i].item()
        pred = predictions[i].item()
        p_val = probs[i][pred].item()

        plt.title(f"GT:{gt} Pred:{pred}\nP:{p_val:.2f}")
        plt.axis('off')

    plt.tight_layout()
    plt.show()
```

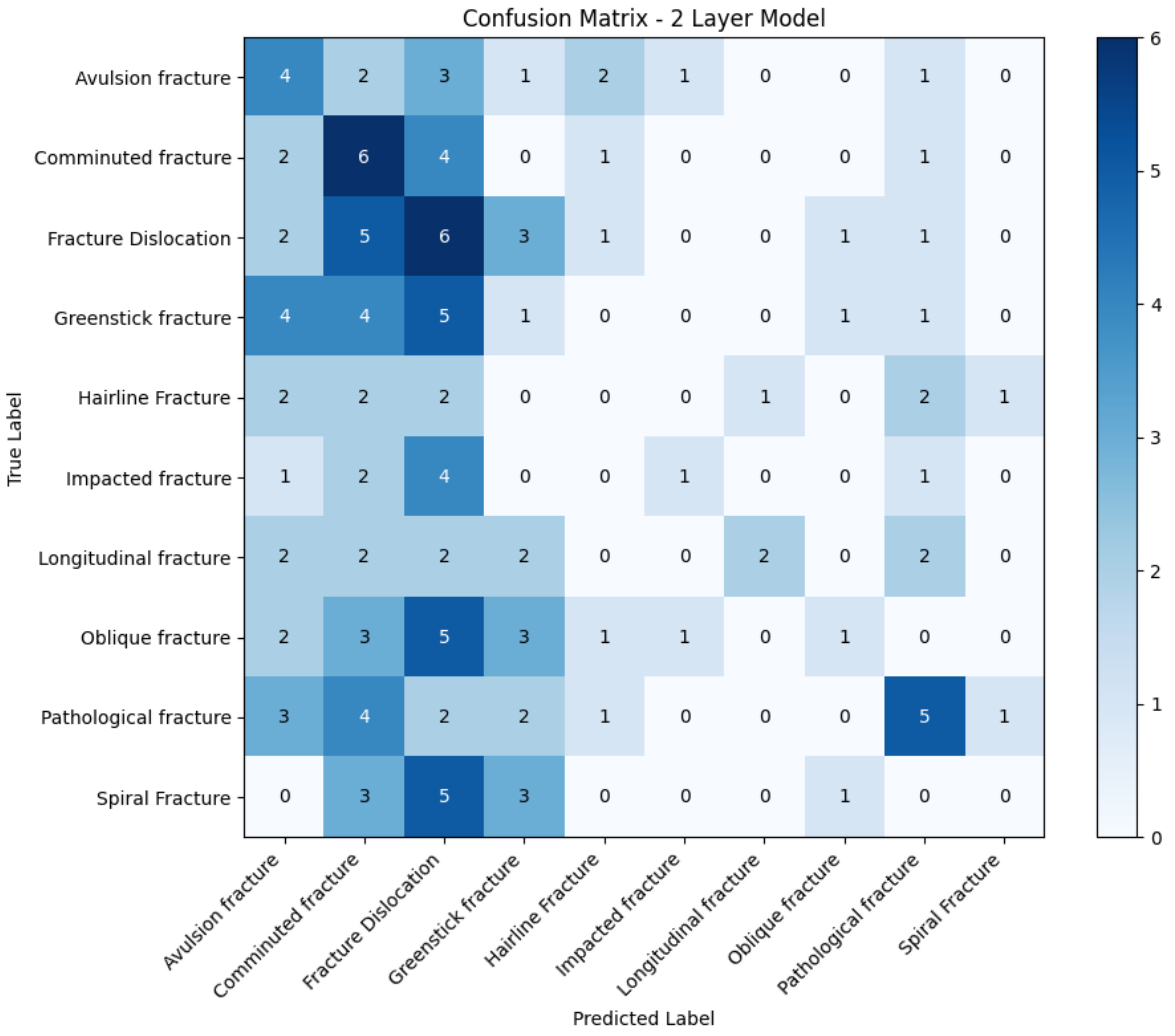
In [17]:
```
visualize_predictions(model, test_loader, device)
```

(8) Make Evaluation with confusion matrix

In [18]:
```
def plot_confusion_matrix(model, data_loader, device, title, figsize=(10,
    model.eval()
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in data_loader:
            images, labels = images.to(device), labels.to(device)
            logits = model(images)
            predictions = torch.argmax(logits, dim=1)
            all_preds.extend(predictions.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    all_preds = np.array(all_preds)
    all_labels = np.array(all_labels)

    cm = confusion_matrix(all_labels, all_preds)

    plt.figure(figsize=figsize)
    plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
    plt.title(title)
    plt.colorbar()

    num_classes = len(np.unique(all_labels))
    tick_marks = np.arange(num_classes)

    if class_names is not None:
        plt.xticks(tick_marks, class_names, rotation=45, ha='right')
        plt.yticks(tick_marks, class_names)
    else:
        plt.xticks(tick_marks, range(num_classes))
```

```
        plt.yticks(tick_marks, range(num_classes))

    thresh = cm.max() / 2.
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(j, i, format(cm[i, j], 'd'),
                     ha="center", va="center",
                     color="white" if cm[i, j] > thresh else "black")

    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.tight_layout()
    plt.show()

    accuracy = np.sum(all_preds == all_labels) / len(all_labels)
    print(f"Test Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")

    return accuracy, cm
```

In [19]:
```
# For first model (2 layers)
accuracy, cm = plot_confusion_matrix(
    model,
    test_loader,
    device,
    title='Confusion Matrix - 2 Layer Model'
)
```




```
Test Accuracy: 0.1857 (18.57%)
```

(9) Instance of a new CNN

Use the Sequential API it is faster

- This cnn has tre hidden layers (conv2d -> proper activation -> max pool 2d) one has 16 untis, the second has 32 units, and the third has 64
- Add a flatten layer
- Classifier need 128 Linear unit proper activation and set dropdown at 0.5
- The last layer of the classifier how many units need?

Repeat from 5 to 8

In [20]:
```
params = {
    "number_of_output_classes":10,
    "kernel_size":3,
    "padding_size":1,
    "pooling_size":2,
    "input_channels":3,
    "dropout_probabilities":0.5,
    "dense_units":[128],
    "layers_list":[16,32,64]
}
model = MyCNN(**params)
model.to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(),lr= 1e-3,weight_decay=1e-4)
```

In [21]:
```
epochs = 50
best_val_acc = 0
val_acc = 0
train_losses, train_accs = [], []
val_losses, val_accs = [], []

for epoch in range (1, epochs + 1):
    train_loss, train_acc = train(model = model,loader=train_loader, crit
    print(f"Epoch #{epoch}/{epochs}, Train Loss = {train_loss:.4f}, Accur
    train_losses.append(train_loss)
    train_accs.append(train_acc)

    val_loss, val_acc = evaluation(model = model,loader=val_loader, crite
    print(f"Epoch #{epoch}/{epochs}, Val Loss = {val_loss:.4f}, Val Accur
    val_losses.append(val_loss)
    val_accs.append(val_acc)

    if (val_acc > best_val_acc):
        best_val_acc = val_acc
        torch.save(model.state_dict(),"Assignment1_best_model_3layer.pt")
print("Best Val Accuracy", best_val_acc)
```

```
Epoch #1/50, Train Loss = 2.4889, Accuracy = 0.096
Epoch #1/50, Val Loss = 2.3030, Val Accuracy = 0.126
Epoch #2/50, Train Loss = 2.2872, Accuracy = 0.137
Epoch #2/50, Val Loss = 2.2772, Val Accuracy = 0.131
Epoch #3/50, Train Loss = 2.2723, Accuracy = 0.135
Epoch #3/50, Val Loss = 2.2649, Val Accuracy = 0.126
Epoch #4/50, Train Loss = 2.2691, Accuracy = 0.135
Epoch #4/50, Val Loss = 2.2552, Val Accuracy = 0.131
Epoch #5/50, Train Loss = 2.2505, Accuracy = 0.154
Epoch #5/50, Val Loss = 2.2544, Val Accuracy = 0.187
Epoch #6/50, Train Loss = 2.2494, Accuracy = 0.153
Epoch #6/50, Val Loss = 2.2509, Val Accuracy = 0.172
Epoch #7/50, Train Loss = 2.2573, Accuracy = 0.162
Epoch #7/50, Val Loss = 2.2467, Val Accuracy = 0.182
Epoch #8/50, Train Loss = 2.2310, Accuracy = 0.198
Epoch #8/50, Val Loss = 2.2520, Val Accuracy = 0.162
Epoch #9/50, Train Loss = 2.2366, Accuracy = 0.171
Epoch #9/50, Val Loss = 2.2507, Val Accuracy = 0.192
Epoch #10/50, Train Loss = 2.2127, Accuracy = 0.183
Epoch #10/50, Val Loss = 2.2195, Val Accuracy = 0.172
Epoch #11/50, Train Loss = 2.2220, Accuracy = 0.180
Epoch #11/50, Val Loss = 2.2257, Val Accuracy = 0.182
Epoch #12/50, Train Loss = 2.1996, Accuracy = 0.204
Epoch #12/50, Val Loss = 2.2159, Val Accuracy = 0.187
Epoch #13/50, Train Loss = 2.2056, Accuracy = 0.186
Epoch #13/50, Val Loss = 2.2211, Val Accuracy = 0.202
Epoch #14/50, Train Loss = 2.1912, Accuracy = 0.192
Epoch #14/50, Val Loss = 2.2124, Val Accuracy = 0.202
Epoch #15/50, Train Loss = 2.1573, Accuracy = 0.200
Epoch #15/50, Val Loss = 2.2205, Val Accuracy = 0.202
Epoch #16/50, Train Loss = 2.1984, Accuracy = 0.182
Epoch #16/50, Val Loss = 2.2257, Val Accuracy = 0.202
Epoch #17/50, Train Loss = 2.1982, Accuracy = 0.205
Epoch #17/50, Val Loss = 2.2192, Val Accuracy = 0.197
Epoch #18/50, Train Loss = 2.1552, Accuracy = 0.205
Epoch #18/50, Val Loss = 2.2188, Val Accuracy = 0.222
Epoch #19/50, Train Loss = 2.1486, Accuracy = 0.220
Epoch #19/50, Val Loss = 2.2078, Val Accuracy = 0.227
Epoch #20/50, Train Loss = 2.1232, Accuracy = 0.249
Epoch #20/50, Val Loss = 2.2132, Val Accuracy = 0.212
Epoch #21/50, Train Loss = 2.1012, Accuracy = 0.216
Epoch #21/50, Val Loss = 2.1952, Val Accuracy = 0.192
Epoch #22/50, Train Loss = 2.0974, Accuracy = 0.234
Epoch #22/50, Val Loss = 2.2090, Val Accuracy = 0.212
Epoch #23/50, Train Loss = 2.1100, Accuracy = 0.231
Epoch #23/50, Val Loss = 2.1988, Val Accuracy = 0.207
Epoch #24/50, Train Loss = 2.0855, Accuracy = 0.234
Epoch #24/50, Val Loss = 2.2214, Val Accuracy = 0.197
Epoch #25/50, Train Loss = 2.0891, Accuracy = 0.240
Epoch #25/50, Val Loss = 2.2067, Val Accuracy = 0.217
Epoch #26/50, Train Loss = 2.0360, Accuracy = 0.258
Epoch #26/50, Val Loss = 2.2004, Val Accuracy = 0.222
Epoch #27/50, Train Loss = 2.0172, Accuracy = 0.249
Epoch #27/50, Val Loss = 2.2110, Val Accuracy = 0.187
Epoch #28/50, Train Loss = 2.0508, Accuracy = 0.260
Epoch #28/50, Val Loss = 2.1992, Val Accuracy = 0.207
Epoch #29/50, Train Loss = 2.0198, Accuracy = 0.283
Epoch #29/50, Val Loss = 2.1981, Val Accuracy = 0.192
Epoch #30/50, Train Loss = 2.0088, Accuracy = 0.250
Epoch #30/50, Val Loss = 2.1991, Val Accuracy = 0.172
```

```
Epoch #31/50, Train Loss = 2.0162, Accuracy = 0.249
Epoch #31/50, Val Loss = 2.2131, Val Accuracy = 0.192
Epoch #32/50, Train Loss = 1.9795, Accuracy = 0.284
Epoch #32/50, Val Loss = 2.2123, Val Accuracy = 0.182
Epoch #33/50, Train Loss = 1.9799, Accuracy = 0.279
Epoch #33/50, Val Loss = 2.2144, Val Accuracy = 0.212
Epoch #34/50, Train Loss = 1.9386, Accuracy = 0.297
Epoch #34/50, Val Loss = 2.2145, Val Accuracy = 0.207
Epoch #35/50, Train Loss = 1.9203, Accuracy = 0.314
Epoch #35/50, Val Loss = 2.2376, Val Accuracy = 0.217
Epoch #36/50, Train Loss = 1.9286, Accuracy = 0.312
Epoch #36/50, Val Loss = 2.2172, Val Accuracy = 0.187
Epoch #37/50, Train Loss = 1.9491, Accuracy = 0.278
Epoch #37/50, Val Loss = 2.2120, Val Accuracy = 0.197
Epoch #38/50, Train Loss = 1.9189, Accuracy = 0.301
Epoch #38/50, Val Loss = 2.2344, Val Accuracy = 0.202
Epoch #39/50, Train Loss = 1.8928, Accuracy = 0.327
Epoch #39/50, Val Loss = 2.2286, Val Accuracy = 0.207
Epoch #40/50, Train Loss = 1.8636, Accuracy = 0.350
Epoch #40/50, Val Loss = 2.2746, Val Accuracy = 0.232
Epoch #41/50, Train Loss = 1.8702, Accuracy = 0.334
Epoch #41/50, Val Loss = 2.2333, Val Accuracy = 0.182
Epoch #42/50, Train Loss = 1.8853, Accuracy = 0.326
Epoch #42/50, Val Loss = 2.2366, Val Accuracy = 0.217
Epoch #43/50, Train Loss = 1.8674, Accuracy = 0.326
Epoch #43/50, Val Loss = 2.2307, Val Accuracy = 0.212
Epoch #44/50, Train Loss = 1.8007, Accuracy = 0.349
Epoch #44/50, Val Loss = 2.2098, Val Accuracy = 0.212
Epoch #45/50, Train Loss = 1.8178, Accuracy = 0.334
Epoch #45/50, Val Loss = 2.2464, Val Accuracy = 0.242
Epoch #46/50, Train Loss = 1.8336, Accuracy = 0.336
Epoch #46/50, Val Loss = 2.2345, Val Accuracy = 0.222
Epoch #47/50, Train Loss = 1.7665, Accuracy = 0.360
Epoch #47/50, Val Loss = 2.3052, Val Accuracy = 0.202
Epoch #48/50, Train Loss = 1.7344, Accuracy = 0.387
Epoch #48/50, Val Loss = 2.3577, Val Accuracy = 0.222
Epoch #49/50, Train Loss = 1.7844, Accuracy = 0.353
Epoch #49/50, Val Loss = 2.2786, Val Accuracy = 0.217
Epoch #50/50, Train Loss = 1.7246, Accuracy = 0.384
Epoch #50/50, Val Loss = 2.3251, Val Accuracy = 0.217
Best Val Accuracy 0.24242424242424243
```

In [22]:
```
model.load_state_dict(torch.load("Assignment1_best_model_3layer.pt"))
print(f"Loaded best model with validation accuracy: {best_val_acc:.4f}")
```

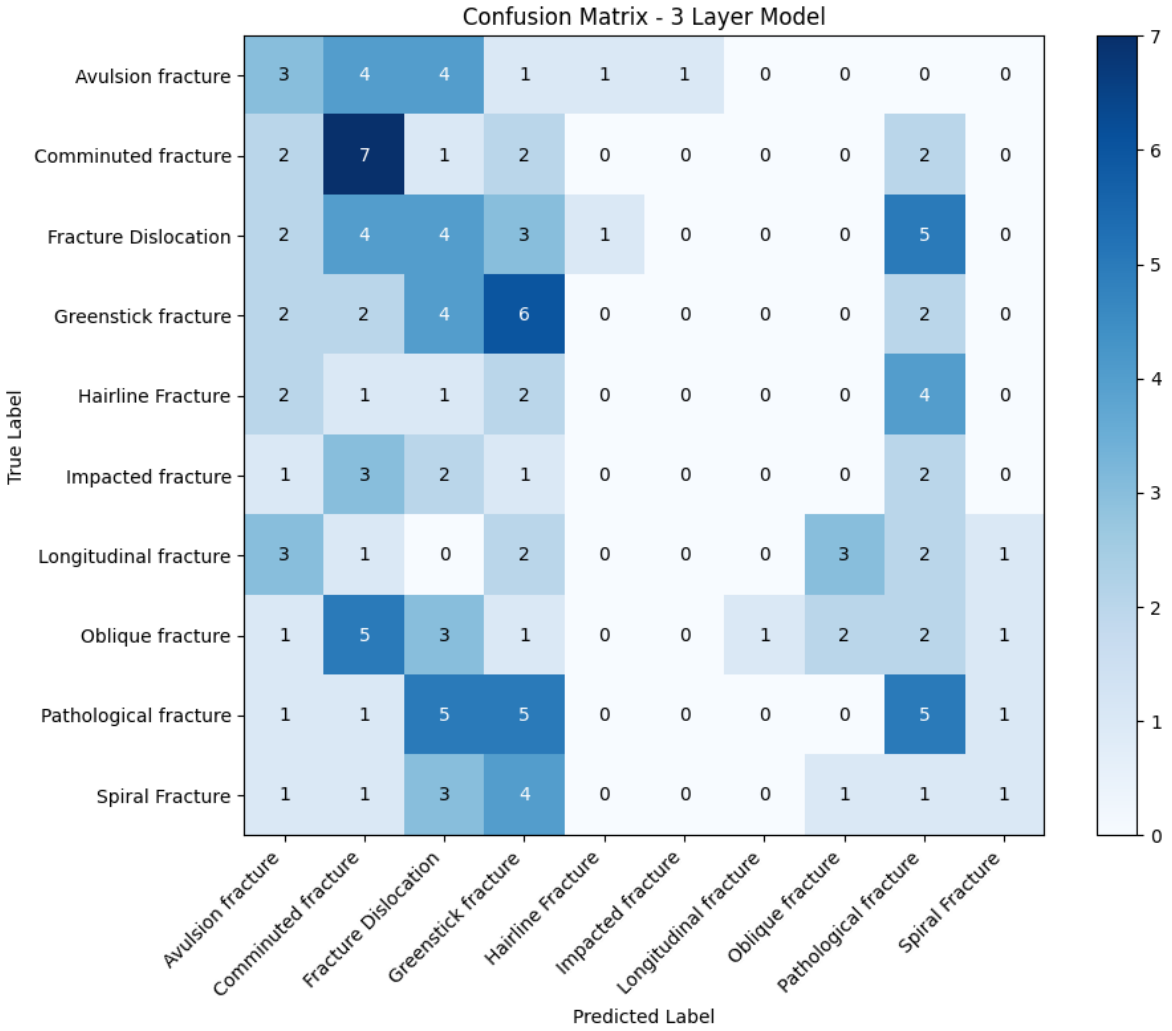
```
Loaded best model with validation accuracy: 0.2424
```

In [23]:
```
plt.figure(figsize=(12, 5))
# Plot 1: Loss Curves
plt.subplot(1, 2, 1)
plt.plot(range(1, len(train_losses) + 1), train_losses, label='Training L
plt.plot(range(1, len(val_losses) + 1), val_losses, label='Validation Los
plt.title('Loss vs. Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
# Plot 2: Accuracy Curves
plt.subplot(1, 2, 2)
plt.plot(range(1, len(train_accs) + 1), train_accs, label='Training Accur
```

```
plt.plot(range(1, len(val_accs) + 1), val_accs, label='Validation Accurac
plt.title('Accuracy vs. Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

In [24]:
```
visualize_predictions(model, test_loader, device)
```

In [25]:
```
# For second model (3 layers)
accuracy, cm = plot_confusion_matrix(
    model,
    test_loader,
    device,
    title='Confusion Matrix - 3 Layer Model'
)
```




```
Test Accuracy: 0.2000 (20.00%)
```

I wanted to play around with the model a bot more. here is that section

In [26]:
```
params = {
    "number_of_output_classes":10,
    "kernel_size":3,
    "padding_size":1,
    "pooling_size":2,
    "input_channels":3,
    "dropout_probabilities":0.1,
    "dense_units":[512],
    "layers_list":[16,32,64]
}
model = MyCNN(**params)
model.to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(),lr= 1e-3,weight_decay=1e-4)
```

In [27]:
```
epochs = 50
best_val_acc = 0
val_acc = 0
train_losses, train_accs = [], []
val_losses, val_accs = [], []

for epoch in range (1, epochs + 1):
    train_loss, train_acc = train(model = model,loader=train_loader, crit
    print(f"Epoch #{epoch}/{epochs}, Train Loss = {train_loss:.4f}, Accur
```

```
    train_losses.append(train_loss)
    train_accs.append(train_acc)

    val_loss, val_acc = evaluation(model = model,loader=val_loader, crite
    print(f"Epoch #{epoch}/{epochs}, Val Loss = {val_loss:.4f}, Val Accur
    val_losses.append(val_loss)
    val_accs.append(val_acc)

    if (val_acc > best_val_acc):
        best_val_acc = val_acc
        torch.save(model.state_dict(),"Assignment1_best_model_3layer.pt")
print("Best Val Accuracy", best_val_acc)
```

```
Epoch #1/50, Train Loss = 3.0651, Accuracy = 0.102
Epoch #1/50, Val Loss = 2.3058, Val Accuracy = 0.111
Epoch #2/50, Train Loss = 2.2987, Accuracy = 0.129
Epoch #2/50, Val Loss = 2.2905, Val Accuracy = 0.177
Epoch #3/50, Train Loss = 2.2739, Accuracy = 0.144
Epoch #3/50, Val Loss = 2.2582, Val Accuracy = 0.162
Epoch #4/50, Train Loss = 2.2482, Accuracy = 0.176
Epoch #4/50, Val Loss = 2.2536, Val Accuracy = 0.182
Epoch #5/50, Train Loss = 2.2313, Accuracy = 0.173
Epoch #5/50, Val Loss = 2.2403, Val Accuracy = 0.187
Epoch #6/50, Train Loss = 2.2212, Accuracy = 0.186
Epoch #6/50, Val Loss = 2.2112, Val Accuracy = 0.177
Epoch #7/50, Train Loss = 2.1831, Accuracy = 0.195
Epoch #7/50, Val Loss = 2.2084, Val Accuracy = 0.202
Epoch #8/50, Train Loss = 2.1801, Accuracy = 0.183
Epoch #8/50, Val Loss = 2.2112, Val Accuracy = 0.187
Epoch #9/50, Train Loss = 2.1530, Accuracy = 0.217
Epoch #9/50, Val Loss = 2.2491, Val Accuracy = 0.177
Epoch #10/50, Train Loss = 2.1602, Accuracy = 0.205
Epoch #10/50, Val Loss = 2.1727, Val Accuracy = 0.197
Epoch #11/50, Train Loss = 2.1007, Accuracy = 0.226
Epoch #11/50, Val Loss = 2.2007, Val Accuracy = 0.182
Epoch #12/50, Train Loss = 2.0749, Accuracy = 0.253
Epoch #12/50, Val Loss = 2.1944, Val Accuracy = 0.182
Epoch #13/50, Train Loss = 2.0516, Accuracy = 0.267
Epoch #13/50, Val Loss = 2.2142, Val Accuracy = 0.182
Epoch #14/50, Train Loss = 2.0314, Accuracy = 0.277
Epoch #14/50, Val Loss = 2.2341, Val Accuracy = 0.182
Epoch #15/50, Train Loss = 2.0011, Accuracy = 0.287
Epoch #15/50, Val Loss = 2.2453, Val Accuracy = 0.182
Epoch #16/50, Train Loss = 1.9996, Accuracy = 0.281
Epoch #16/50, Val Loss = 2.2285, Val Accuracy = 0.217
Epoch #17/50, Train Loss = 1.9902, Accuracy = 0.295
Epoch #17/50, Val Loss = 2.2476, Val Accuracy = 0.146
Epoch #18/50, Train Loss = 1.9370, Accuracy = 0.317
Epoch #18/50, Val Loss = 2.2594, Val Accuracy = 0.197
Epoch #19/50, Train Loss = 1.9197, Accuracy = 0.307
Epoch #19/50, Val Loss = 2.2826, Val Accuracy = 0.187
Epoch #20/50, Train Loss = 1.9048, Accuracy = 0.330
Epoch #20/50, Val Loss = 2.3002, Val Accuracy = 0.192
Epoch #21/50, Train Loss = 1.8818, Accuracy = 0.325
Epoch #21/50, Val Loss = 2.2936, Val Accuracy = 0.212
Epoch #22/50, Train Loss = 1.8572, Accuracy = 0.327
Epoch #22/50, Val Loss = 2.2942, Val Accuracy = 0.207
Epoch #23/50, Train Loss = 1.8296, Accuracy = 0.364
Epoch #23/50, Val Loss = 2.2988, Val Accuracy = 0.212
Epoch #24/50, Train Loss = 1.8516, Accuracy = 0.364
Epoch #24/50, Val Loss = 2.3018, Val Accuracy = 0.187
Epoch #25/50, Train Loss = 1.7466, Accuracy = 0.403
Epoch #25/50, Val Loss = 2.2503, Val Accuracy = 0.222
Epoch #26/50, Train Loss = 1.7565, Accuracy = 0.388
Epoch #26/50, Val Loss = 2.3491, Val Accuracy = 0.227
Epoch #27/50, Train Loss = 1.7295, Accuracy = 0.387
Epoch #27/50, Val Loss = 2.3211, Val Accuracy = 0.217
Epoch #28/50, Train Loss = 1.6703, Accuracy = 0.407
Epoch #28/50, Val Loss = 2.3785, Val Accuracy = 0.247
Epoch #29/50, Train Loss = 1.6367, Accuracy = 0.431
Epoch #29/50, Val Loss = 2.4366, Val Accuracy = 0.212
Epoch #30/50, Train Loss = 1.6369, Accuracy = 0.435
Epoch #30/50, Val Loss = 2.4321, Val Accuracy = 0.212
```

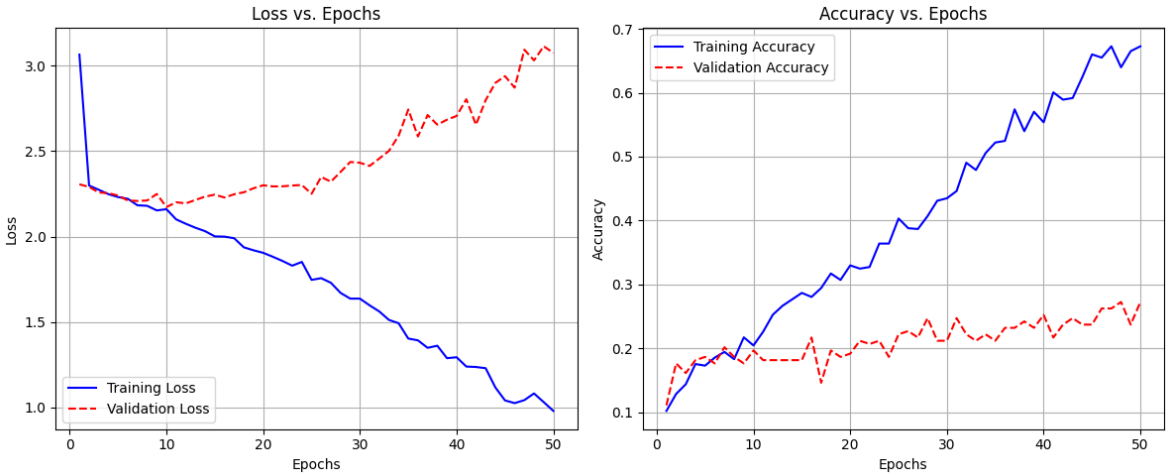
```
Epoch #31/50, Train Loss = 1.5976, Accuracy = 0.446
Epoch #31/50, Val Loss = 2.4134, Val Accuracy = 0.247
Epoch #32/50, Train Loss = 1.5618, Accuracy = 0.491
Epoch #32/50, Val Loss = 2.4567, Val Accuracy = 0.222
Epoch #33/50, Train Loss = 1.5122, Accuracy = 0.479
Epoch #33/50, Val Loss = 2.5013, Val Accuracy = 0.212
Epoch #34/50, Train Loss = 1.4928, Accuracy = 0.506
Epoch #34/50, Val Loss = 2.5900, Val Accuracy = 0.222
Epoch #35/50, Train Loss = 1.4031, Accuracy = 0.522
Epoch #35/50, Val Loss = 2.7437, Val Accuracy = 0.212
Epoch #36/50, Train Loss = 1.3925, Accuracy = 0.525
Epoch #36/50, Val Loss = 2.5859, Val Accuracy = 0.232
Epoch #37/50, Train Loss = 1.3488, Accuracy = 0.574
Epoch #37/50, Val Loss = 2.7114, Val Accuracy = 0.232
Epoch #38/50, Train Loss = 1.3612, Accuracy = 0.540
Epoch #38/50, Val Loss = 2.6549, Val Accuracy = 0.242
Epoch #39/50, Train Loss = 1.2883, Accuracy = 0.570
Epoch #39/50, Val Loss = 2.6839, Val Accuracy = 0.232
Epoch #40/50, Train Loss = 1.2936, Accuracy = 0.554
Epoch #40/50, Val Loss = 2.7060, Val Accuracy = 0.253
Epoch #41/50, Train Loss = 1.2393, Accuracy = 0.601
Epoch #41/50, Val Loss = 2.8037, Val Accuracy = 0.217
Epoch #42/50, Train Loss = 1.2367, Accuracy = 0.589
Epoch #42/50, Val Loss = 2.6542, Val Accuracy = 0.237
Epoch #43/50, Train Loss = 1.2296, Accuracy = 0.592
Epoch #43/50, Val Loss = 2.7984, Val Accuracy = 0.247
Epoch #44/50, Train Loss = 1.1173, Accuracy = 0.625
Epoch #44/50, Val Loss = 2.8993, Val Accuracy = 0.237
Epoch #45/50, Train Loss = 1.0411, Accuracy = 0.660
Epoch #45/50, Val Loss = 2.9390, Val Accuracy = 0.237
Epoch #46/50, Train Loss = 1.0251, Accuracy = 0.655
Epoch #46/50, Val Loss = 2.8715, Val Accuracy = 0.263
Epoch #47/50, Train Loss = 1.0427, Accuracy = 0.673
Epoch #47/50, Val Loss = 3.0937, Val Accuracy = 0.263
Epoch #48/50, Train Loss = 1.0818, Accuracy = 0.640
Epoch #48/50, Val Loss = 3.0312, Val Accuracy = 0.273
Epoch #49/50, Train Loss = 1.0312, Accuracy = 0.665
Epoch #49/50, Val Loss = 3.1136, Val Accuracy = 0.237
Epoch #50/50, Train Loss = 0.9795, Accuracy = 0.673
Epoch #50/50, Val Loss = 3.0749, Val Accuracy = 0.273
Best Val Accuracy 0.2727272727272727
```

In [28]:
```
model.load_state_dict(torch.load("Assignment1_best_model_3layer.pt"))
print(f"Loaded best model with validation accuracy: {best_val_acc:.4f}")
```

```
Loaded best model with validation accuracy: 0.2727
```

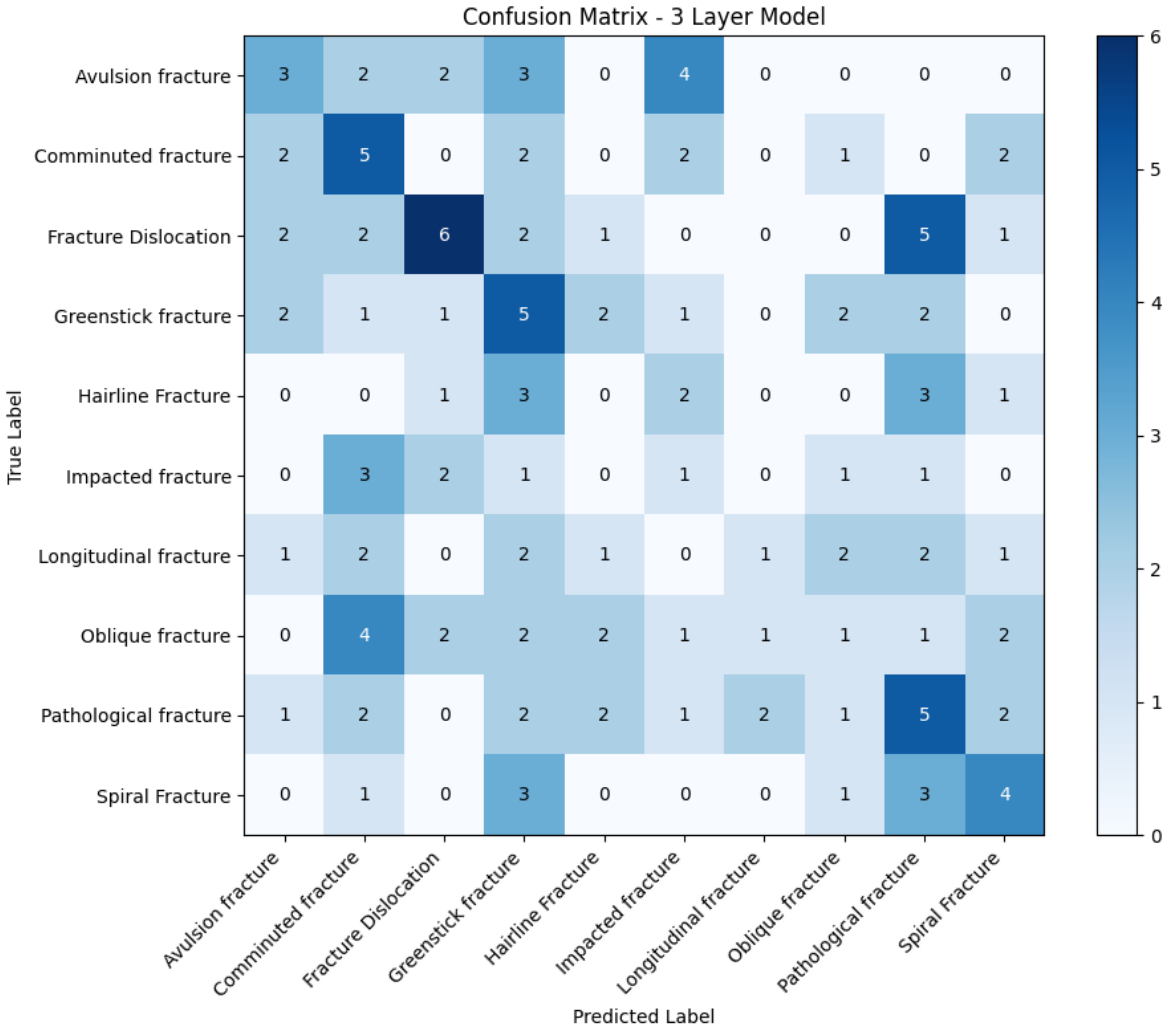
In [29]:
```
plt.figure(figsize=(12, 5))
# Plot 1: Loss Curves
plt.subplot(1, 2, 1)
plt.plot(range(1, len(train_losses) + 1), train_losses, label='Training L
plt.plot(range(1, len(val_losses) + 1), val_losses, label='Validation Los
plt.title('Loss vs. Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
# Plot 2: Accuracy Curves
plt.subplot(1, 2, 2)
plt.plot(range(1, len(train_accs) + 1), train_accs, label='Training Accur
```

```
plt.plot(range(1, len(val_accs) + 1), val_accs, label='Validation Accurac
plt.title('Accuracy vs. Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



In [30]:
```
visualize_predictions(model, test_loader, device)
```

In [31]:
```
# For second model (3 layers) but modified parameters
accuracy, cm = plot_confusion_matrix(
    model,
    test_loader,
    device,
    title='Confusion Matrix - 3 Layer Model'
)
```




```
Test Accuracy: 0.2214 (22.14%)
```